

Ingeniería de Software I

Arquitectura de servicios

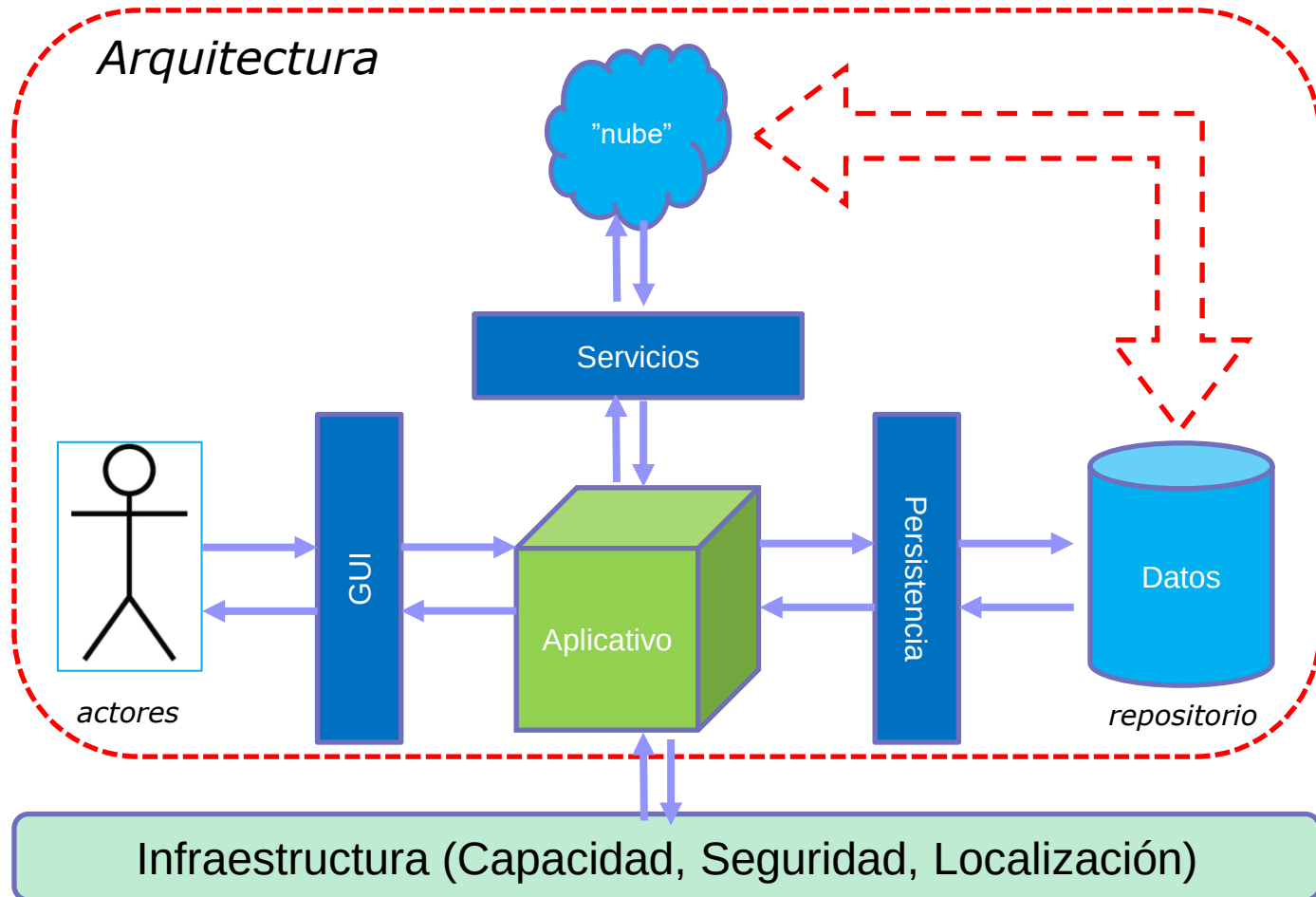
Docente: Dr. Pedro E. Colla

Diseño orientado a servicios

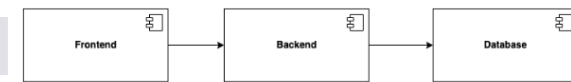


Interfaces

Los aplicativos tienen típicamente tres interfaces



La identificación y diseño de las mismas son parte de la arquitectura

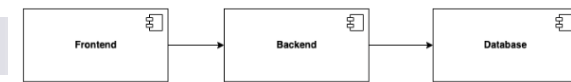


Service Oriented Architecture (SOA)

Describe un método estándar para requerir y entregar servicios en un ambiente de componentes distribuidos y como entregar resultados

- El foco principal es en las características de la interfaz del servicio y las características del mismo.
- Provee una capa de traducción y gestión que remueve las dificultades para lograr cooperación entre servicios independientemente de su tecnología de implementación.
- Permite la utilización de diferentes arquitecturas de red y protocolos de mensajes.

Al conjunto de servicios en un aplicativo se lo denomina “bus”



SOA (Ventajas)

Implementar funciones mediante SOA habilita algunas ventajas respecto a su implementación en forma monolítica

Reducción del plazo de desarrollo

Los desarrolladores pueden reutilizar servicios en diferentes procesos empresariales. Pueden crear aplicaciones en menos tiempo con SOA en lugar de escribir código y llevar a cabo integraciones desde cero.

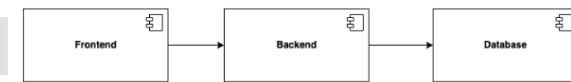
Mantenimiento eficiente

Es más fácil crear, actualizar y corregir errores en servicios pequeños que en bloques grandes de código en aplicaciones monolíticas. La modificación de un servicio en SOA no afecta a la funcionalidad general del proceso empresarial.

Capacidad de adaptación

El bajo acoplamiento entre componentes permite su adaptación y reemplazo.

La naturaleza y características de la comunicación entre componentes dictará cual es el patrón de arquitectura mas conveniente



SOA Criterios básicos

No existen estándares establecidos de implementación de SOA, sin embargo hay algunos principios rectores que deben cuidarse

Interoperabilidad

Cada servicio de describir la funcionalidad así como las condiciones de provisión de servicios. Esto aísla la interacción de la implementación.

Acoplamiento flexible

Los servicios deben tener poca dependencia en recursos externos, como modelos de datos o sistemas de información.

Stateless

Deben prescindir de un estado sin retener ninguna información sobre sesiones o transacciones pasadas.

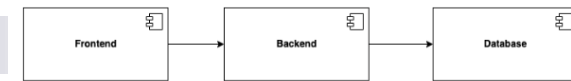
Abstracción

No deben conocer la lógica o detalles de implementación. Sus detalles se obtienen mediante un “contrato de servicio”.

Granularidad

Se debe especializar los servicios para que tengan un tamaño manejable.

La naturaleza y características de la comunicación entre componentes dictará cual es el patrón de arquitectura mas conveniente



SOA Componentes básicos

Los componentes básicos son servicio, proveedor, consumidor y registro

Servicio

Son los componentes básicos, pueden ser privados o públicos. Se definen por:

- *Implementación de servicios*
- *Contrato del servicio*
- *Interfaz del servicio*

Proveedor de servicios

El proveedor de servicios crea, mantiene y proporciona uno o más servicios.

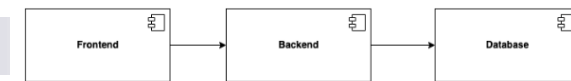
Consumidor de servicios

El consumidor de servicios solicita al proveedor un servicio específico.

Registro o Repositorio de servicio

Es un directorio de servicios disponibles accesible a través de redes.

La naturaleza y características de la comunicación entre componentes dictará cual es el patrón de arquitectura mas conveniente



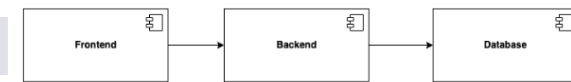
SOA Protocolos de comunicación

Los servicios se comunican mediante reglas establecidas que determinan la transmisión de datos en una red. Estas reglas se llaman protocolos de comunicación.

Algunos protocolos estándar para implementar SOA incluyen los siguientes:

- Protocolo simple de acceso a objetos (SOAP)
- REST HTTP
- Apache Thrift
- Apache ActiveMQ
- Servicio de mensajes de Java (JMS)
- MQ-Series (IBM).

Se puede utilizar mas de un protocolo en la implementación de un servicio dependiendo de las necesidades



Contrato de servicio

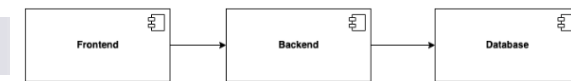
Se denomina contrato a la descripción de la naturaleza y condiciones de uso de un servicio.

WSDL (Web Services Description Language)

- Es una definición de interface basada en XML.
- Consiste en tres elementos: Type, Binding y Operations.
- Soporta varios métodos de comunicación (REST, SOAP, MIME).

<https://www.w3.org/TR/wsdl.html>

Puede ser utilizado tanto para el diseño como para la comunicación de los contratos “honrados” por un determinado servicio



Microservicios

Los microservicios son componentes de propósito único, pequeños y completamente independientes de otros servicios a los que se accede por medio de una API definida. Contienen sus propios datos por lo que se evita el problema de recursos compartidos y puede ser multiplicado para obtener escalabilidad y redundancia.

Problemas de SOA

- **Escalabilidad limitada**

La escalabilidad de sistemas se ve seriamente afectada cuando los servicios comparten varios recursos y necesitan coordinarse para realizar su respectiva funcionalidad.

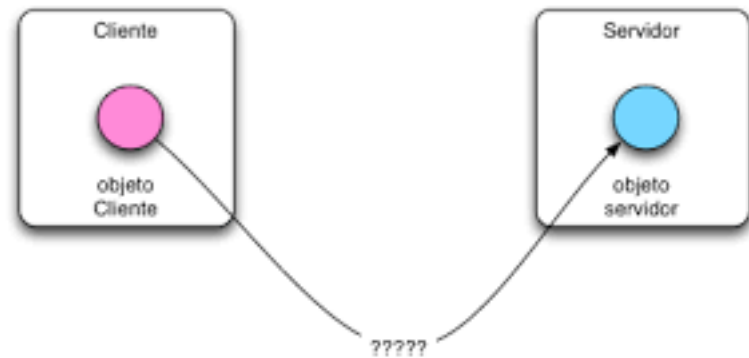
- **Aumento en las interdependencias**

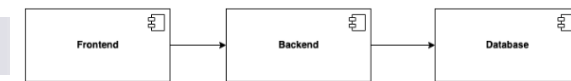
Aparición de bucles de componentes y recursos compartidos.

- **Punto único de falla**

Los microservicios abordan los inconvenientes de la implementación utilizando un enfoque SOA puro

Protocolos Servicios





REST

REST (**R**epresentational **S**tate **T**ransfer) permite la interacción de componentes distribuidos utilizando una infraestructura Web

Es un mecanismo que encapsula los requerimientos de interfaz de un componente como un requerimiento HTTP (GET o POST).

Obtiene como resultado información expresada en un formato predefinido, usualmente estructurada como alguna variante basada en XML o JSON.

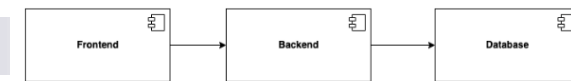
Ventajas

- Simplicidad e independencia del lenguaje.
- Escalabilidad

Desventajas

- No hay contrato.
- No mantiene el concepto de sesión (una llamada por conexión).

Uso en API publicas y servidores con poca capacidad. Apps móviles.



REST (Ejemplo)



Request: /pets

Status: 200

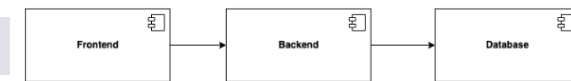
Latency: 566 ms

Response Body

```
{
  "pet": {
    "type": "dog",
    "price": 249.99
  },
  "message": "success"
}
```

Response Headers

```
{"Access-Control-Allow-Origin":"*","X-Amzn-Trace-Id":"Root=1-59287e14-bd5f1d07c673367be0739eae","Content-Type":"application/json"}
```



REST (Ejemplo)



HTTP (GET)

Los argumentos son parte del URL

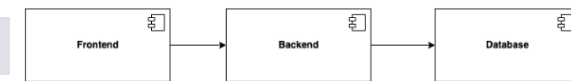
<http://www.myserver.com/pets?raza=caniche?color=blanco>

HTTP (POST)

Los argumentos son parte de un mensaje separado del URL que se transmite a continuación junto con "tags" (marcas) indicando la naturaleza y codificación del mismo.

<http://www.myserver.com/pets>

```
{"pet": {  
  "raza": "caniche",  
  "color": "blanco"}  
}
```



REST (Ejemplo)



Ventajas de GET

Los parámetros URL se pueden guardar junto a la dirección URL como marcador. De esta manera, puedes introducir una búsqueda y más tarde consultarla de nuevo fácilmente. También se puede volver a acceder a la página a través del historial del navegador.

Es mas práctico para operación manual de una invocación HTTP pues permite fácilmente editar los parámetros de la llamada.

Desventajas de GET

La mayor desventaja del método GET es su débil protección de los datos. Los parámetros URL que se envían quedan visibles en la barra de direcciones del navegador y son accesibles sin clave en el historial de navegación, en el caché y en el log de los servidores.

También su capacidad es limitada: dependiendo del servidor y del navegador, no es posible introducir más de 2000 caracteres. Es necesario, además, utilizar caracteres de "escape" para poder incluir símbolos que no sean ASCII imprimibles.

REST (Ejemplo)



Ventajas de POST

Mejor desde el punto de vista seguridad pues los datos no están visibles. Es posible además de texto enviar otros tipos de información, como fotos o vídeos codificados en forma conveniente (por ejemplo BASE64).

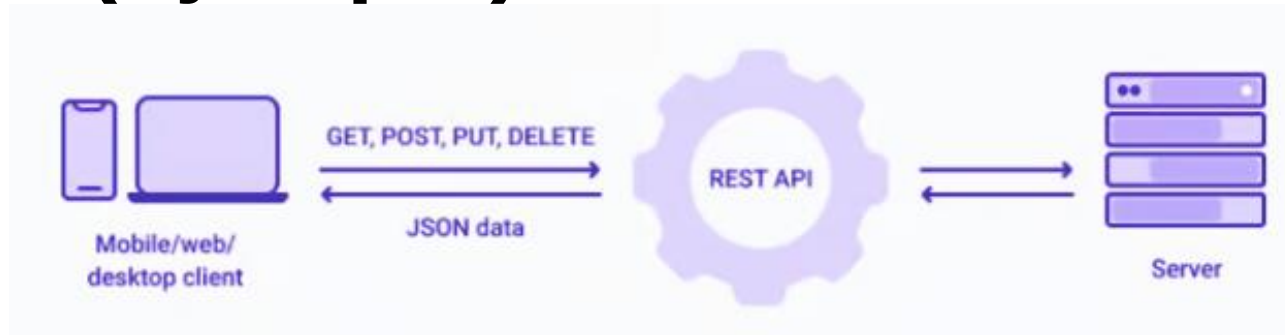
Desventajas de POST

No permite navegación manual sin enviar nuevamente la información.

Los datos transferidos con el método POST no pueden guardarse junto al URL como marcador.

En general el método POST es mayormente indicado para interacción programática de requerimientos HTTP.

REST (Ejemplo)



HEAD:

entregará las respuestas de los encabezados y no las respuestas específicas.

PUT:

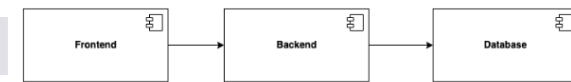
método que se encarga de modificar datos.

DELETE:

método que elimina algún dato.

PATCH:

método que modifica, de forma parcial, los datos.



JSON

JavaScript Object Notation (JSON) es un formato de datos popular entre los desarrolladores debido a que su texto es legible por humanos, es ligero, requiere menos codificación y se procesa más rápido.

Los datos están separados por comas.

Los datos se escriben en pares, siendo primero el nombre o atributo del mismo y luego el valor del dato.

Los objetos JSON están rodeados por llaves “{ }”

Llaves cuadradas [] guardan arreglos, incluyendo otros objetos JSON.

Originalmente utilizados en Java pero se los considera independientes del lenguaje en éste momento y extremadamente flexibles

REST API Emulator

Presentation

Application

Business

Persistence

Database

Es útil durante el proceso para el diseño emular al REST API

My JSON Server

Fake Online REST server for teams

Create a **JSON file** on **GitHub**

Get **instantly** a **fake server**

github.com/user/repo/main/db.json

```
{
  "posts": [
    {
      "id": 1,
      "title": "hello"
    }
  ],
  "profile": {
    "name": "typicode"
  }
}
```

my-json-server.typicode.com/user/repo/posts/1

```
{
  "id": 1,
  "title": "hello"
}
```

Preferencias del Sistema

REST API Emulator

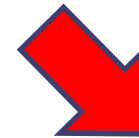
Permite generar registros editados manualmente y usar REST API

<https://my-json-server.typicode.com/lu7did/mesaayuda/posts/803a62c8-78c8-4b63-9106-73af216d504b>

```

1  {
2    "posts": [
3      {
4        "nombre": "Dr. Pedro E. Colla",
5        "contacto": "colla.pedro@uader.edu.ar",
6        "password": "secret",
7        "fecha_ultimo_ingreso": "11/08/2024",
8        "activo": true,
9        "fecha_cambio_password": "01-01-2023",
10       "registrado": true,
11       "primer_ingreso": false,
12       "fecha_alta": "01-01-2023",
13       "id": "803a62c8-78c8-4b63-9106-73af216d504b"
14     },
15     {
16       "nombre": "Dr. Pedro E. Colla",
17       "contacto": "pcolla@iua.edu.ar",
18       "password": "secret",
19       "fecha_ultimo_ingreso": "13-11-2023",
20       "fecha_cambio_password": "01-01-2023",
21       "activo": true,
22       "registrado": true,
23       "primer_ingreso": false,
24       "fecha_alta": "01-01-2023",
25       "id": "666-62c8-78c8-4b63-9106-73af216d504b"
26     },

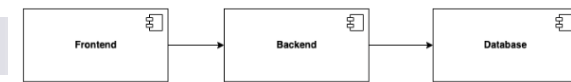
```



```

{
  "nombre": "Dr. Pedro E. Colla",
  "contacto": "colla.pedro@uader.edu.ar",
  "password": "secret",
  "fecha_ultimo_ingreso": "11/08/2024",
  "activo": true,
  "fecha_cambio_password": "01-01-2023",
  "registrado": true,
  "primer_ingreso": false,
  "fecha_alta": "01-01-2023",
  "id": "803a62c8-78c8-4b63-9106-73af216d504b"
}

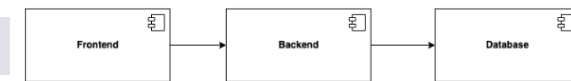
```



SOAP

Services Oriented Architecture Protocol (SOAP) un protocolo de mensajes basado en XML

- Tiene cuatro capas: Header, Body, Envelope y Fault.
- Trabaja “montado” en un protocolo de capa tipo HTTP o SMTP
- Sus principales características son:
 - Independencia del lenguaje.
 - Neutralidad de arquitectura.
 - Extensibilidad.
- **Ventajas**
 - Puede usar múltiples protocolos.
 - Establece un contrato claro entre el requerimiento y la respuesta.
- **Desventajas**
 - Mas difícil para desarrollar.
 - Mas volumen/Menos performance que REST.



SOAP

Request

```
POST /StockQuote HTTP/1.1 Host: www.stockquoteserver.com
Content-Type: text/xml;
charset="utf-8"
Content-Length: nnnn
SOAPAction: "Some-URI"
<?xml version="1.0"?>
<SOAP-ENV:Envelope xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/"
SOAP-ENV:encodingStyle="http://schemas.xmlsoap.org/soap/encoding/">
  <SOAP-ENV:Body> <m:GetLastTradePrice xmlns:m="Some-URI">
    <symbol>DIS</symbol>
  </m:GetLastTradePrice>
</SOAP-ENV:Body>
</SOAP-ENV:Envelope>
```

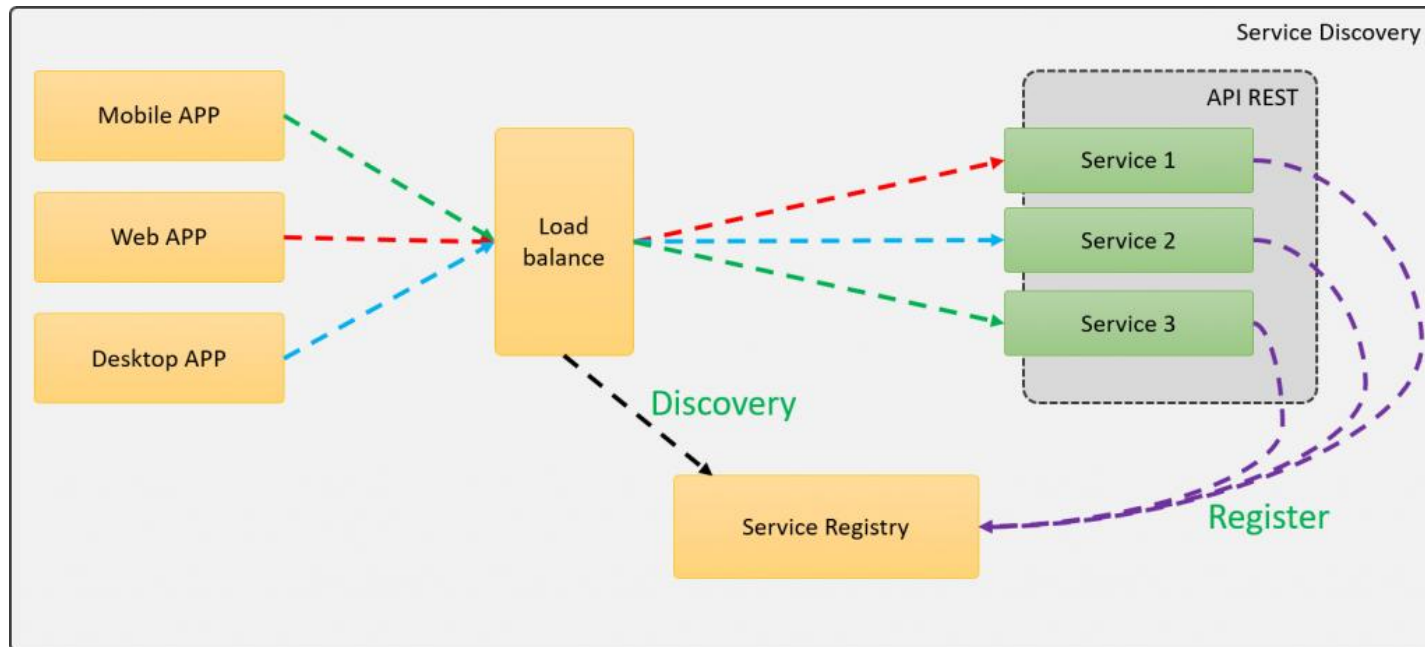
Response

```
HTTP/1.1 200 OK
Content-Type: text/xml;
charset="utf-8"
Content-Length: nnnn
<?xml version="1.0"?>
<SOAP-ENV:Envelope xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/"
SOAP-ENV:encodingStyle="http://schemas.xmlsoap.org/soap/encoding/">
  <SOAP-ENV:Body>
    <m:GetLastTradePriceResponse xmlns:m="Some-URI">
      <Price>34.5</Price>
    </m:GetLastTradePriceResponse>
  </SOAP-ENV:Body>
</SOAP-ENV:Envelope>
```

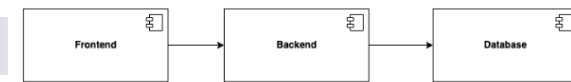


Arquitectura REST típica

Cuando la demanda excede a un solo servidor es necesario incrementar los servidores disponibles y utilizar un componente llamado “*load balancer*” (balanceador de carga)



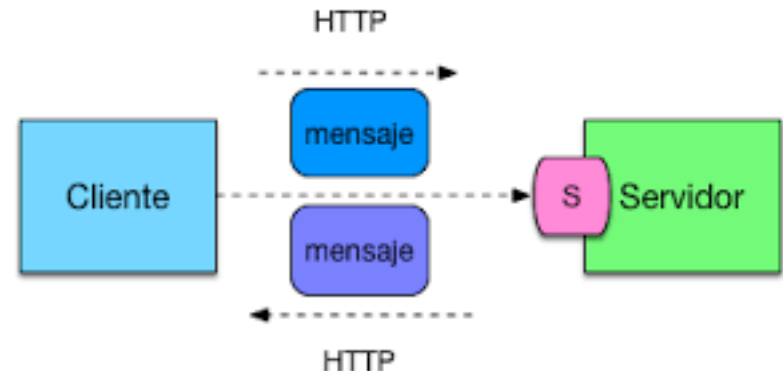
El registro de servicio es el componente que permite mantener la persistencia entre múltiples interacciones entre las APP y el servidor



Arquitectura REST típica

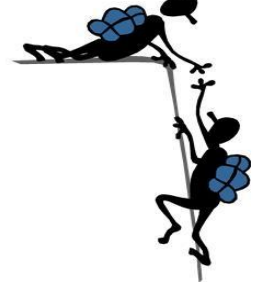
Una parte significativa del esfuerzo de despliegue de la arquitectura está relacionado con desplegar los componentes de infraestructura, servicios periféricos y gestionar la capacidad.

- Dificultades para gestionar la capacidad.
- Necesidad de gestionar la operación de los servidores.
- Necesidad de desplegar servidores adicionales (datos, seguridad).
- El costo de licenciamiento y gestión operativo puede ser substancial.
- Lenguajes típicos:
 - Python.
 - Java.
 - Node.js (JavaScript modificado).
 - C#.
 - Otros.



Debe desplegarse un motor de base de datos para persistencia

Taller



Implemente un servidor de aplicaciones local en NodeJS que responda a comandos básicos requeridos para las funciones a implementar en Mesa de Ayuda.

- Despliegue el servidor NodeJS como infraestructura.
- Despliegue el aplicativo que implementa el API.
- Realice el test del ambiente utilizando Postman.

Utilice los lineamientos especificados en la siguiente nota de aplicación

IS1_NAPP_Ambiente Desarrollo Local NodeJS.pdf

Taller



- Implemente un servidor NodeJS trivial que acepte requerimientos tipo GET o POST sin argumentos, al recibirlos debe responder con el texto “Hello World”.
 - Implemente el código NodeJS del servidor.
 - Analice su estructura.
 - Pruebe el servidor mediante Postman o Insomnia (API client).
 - Realice modificaciones triviales al servidor y verifique.
- Repita lo anterior pero utilizando los servicios de la librería NodeJS Express especializada en utilizar REST API

IS1_Taller_HelloWorld.js
IS1_Taller_Express.js

Taller

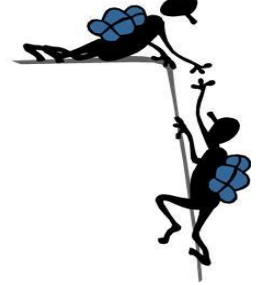


- Tomando el caso de la mesa de ayuda desarrollado en clases anteriores. Se desea implementar como un servicio que utilice GET/POST para retornar un número específico de cliente denominado, provisto su **id**.
- Diseñe un formato API REST/GET para requerir el servicio.
- Diseñe el formato JSON que implementará la respuesta.
- Realice una validación preliminar sin acceder a una base de datos real utilizando un “falso” REST API server

IS1_Taller_RESTDemo.js

Utilizar id de cliente 803a62c8-78c8-4b63-9106-73af216d504b

Taller



El diseño del servicio SOA (API REST) luce de la siguiente forma

Requerimiento al servidor (GET)

http://{url servidor}:8080/cliente?id=XXXXXXXXXXXXXXXXXXXXXXXXXXXX

Respuesta del servidor (JSON)

Respuesta correcta

```
{
  "response": "OK",
  "nombre": "Dr. Pedro E. Colla",
  "contacto": "colla.pedro@uader.edu.ar",
  "password": "secret",
  "fecha_ultimo_ingreso": "11/08/2024",
  "activo": true,
  "fecha_cambio_password": "01-01-2023",
  "registrado": true,
  "primer_ingreso": false,
  "fecha_alta": "01-01-2023",
  "id": "803a62c8-78c8-4b63-9106-73af216d504b"
}
```

Respuesta incorrecta o error

```
{
  "response": "ERROR",
  "id": "803a62c8-78c8-4b63-9106-73af216d504b"
}
```

¿Preguntas?

